

ALCUNE REGOLE D'ORO PER L'ESAME DI TDP

Riassunto: La Guida Definitiva per l'Esame

Stampati a mente questo schemino, copre il 100% delle domande sui grafi del tuo esame:

1. Ti chiedono il **cammino più corto (meno archi/salti)?**

Scegli la **BFS** (in Python basta usare `nx.shortest_path(G, source, target)` che fa la BFS per te).

2. Ti chiedono il **cammino meno costoso (peso minimo)?**

Scegli **Dijkstra** (in Python usi `nx.dijkstra_path(G, source, target, weight='weight')`).

3. Ti chiedono il **cammino più lungo**, oppure di **massimizzare** qualcosa, oppure c'è una regola strana (es. "peso decrescente")?

Scegli la **DFS / Ricorsione**

ESAME 12/01/2026

FARE ATTENZIONE A:

ERRORE NELLE DATACLASS (TypeError: takes 1 positional argument...)

All'esame dovrai creare quasi sicuramente una classe per i Nodi e una per gli Archi.

- Perché hai sbagliato: Hai usato il segno uguale (=). In Python, se usi =, stai assegnando un valore fisso alla classe. Quindi Python pensa: *"Ah, queste variabili sono già fisse, non devo chiederle quando creo un nuovo arco!"* e va in crash quando cerchi di passargli i dati dal database.
- Come deve essere all'esame: Usa SEMPRE i due punti (:).

● SBAGLIATO:

```
@dataclass
class Arco:
    id1 = Constructor # ERRORE GRAVISSIMO
    peso = int
```

● CORRETTO:

```
@dataclass
class Arco:
    id1: Constructor # CORRETTO
    peso: int
```

ERRORE NEL DAO NODI (AttributeError: 'int' object has no attribute 'name')

Questo è l'errore che ti ha fatto perdere più tempo in assoluto.

- **Perché hai sbagliato:** Nella tua SELECT dei Nodi estraevi solo l'ID (SELECT constructorId) e aggiungevi alla lista solo quello. Nel resto del programma ti portavi dietro dei banali **numeri** (es. 128), e quando l'interfaccia provava a stampare `nodo.name`, Python andava in crash perché il numero 128 non ha un nome.
- **Come deve essere all'esame:** Il DAO Nodi deve **SEMPRE** creare Oggetti completi, non liste di numeri, a meno che non mi serva solo quel numero

● SBAGLIATO (Nel DAO):

```
codePython

query = "SELECT constructorId FROM constructors ..."

# ...

for row in cursor:

    results.append(row["constructorId"])

# Hai creato una lista di numeri!
```

● CORRETTO (Nel DAO):

```
codePython

query = "SELECT id, name, constructorRef FROM
constructors ..."

# ...

for row in cursor:

    results.append(Constructor(**row))

# Hai creato una lista di OGGETTI!
```

ERRORE NELLA MAPPA (Stampa numeri invece che nomi)

La Mappa (_idMap) serve per tradurre velocemente un ID nel suo Oggetto corrispondente quando crei gli Archi.

- **Perché hai sbagliato:** Creavi la mappa associando l'ID all'ID (_idMap[128] = 128). Questo rendeva la mappa completamente inutile.
- **Come deve essere all'esame:** La chiave è l'ID, ma **il valore deve essere l'intero oggetto n.**

● SBAGLIATO (Nel Model, quando crei il grafo):

```
codePython

for n in self._nodes:

    self._idMap[n.constructorId] = n.constructorId
```

● CORRETTO (Nel Model):

```
codePython

for n in self._nodes:

    self._idMap[n.constructorId] = n # CORRETTO

#salvi l'oggetto intero
```

ERRORE NEL CONTROLLER (Doppia traduzione inutile)

- **Perché hai sbagliato:** Nel file controller.py, quando iteravi sui risultati di NetworkX, provavi a usare di nuovo _idMap per tradurre le variabili, pensando fossero ID.
- **Come deve essere all'esame:** NetworkX ti restituisce esattamente quello che gli hai dato. Se nel grafo ci hai messo Oggetti, lui ti restituisce Oggetti. Nel Controller non devi MAI usare _idMap.

● SBAGLIATO (Nel Controller):

```
codePython

for u, v, dati in top3:

    nodo1 = self._model._idMap[u] # ERRORE:

#u è già un oggetto! Crasha!

stampa(nodo1.name)
```

● CORRETTO (Nel Controller):

```
codePython

for u, v, dati in top3:

    # u e v sono GIÀ Oggetti Constructor completi.

    Usali direttamente!

    stampa(f"{u.name} -> {v.name}")
```

ESAME SIMULAZIONE1

FARE ATTENZIONE A:

Confondere gli "ID" con gli "Oggetti"

Questa è la cosa su cui hai avuto l'errore logico più subdolo.

- Cosa è successo: Quando creavi gli archi, stavi estraendo le tuple dal DAO (es. (14, 52)) e stavi dicendo a NetworkX di collegare il numero 14 al numero 52 (self._graph.add_edge(id1, id2)).
- Perché è complicato: NetworkX è "stupido": se gli dici di collegare due numeri, lui crea due nodi invisibili fatti di numeri e li collega, sballandoti tutto il grafo. Non ti dà un errore in rosso, semplicemente i numeri a fine esecuzione sono sballati.

```
def buildGraph(self, genere):
```

```
    # ... creazione iniziale del grafo
    mappa_popolarita = DAO.getPopolarita(genere)
    archi = DAO.getAllEdges(genere)
    for id1, id2 in archi:
        # ● ERRORE 1: Assegnazione Sbagliata
        a = mappa_popolarita[id1]
        b = mappa_popolarita[id2]

        pop_a = mappa_popolarita.get(id1, 0)
        pop_b = mappa_popolarita.get(id2, 0)
        peso = pop_a + pop_b

        if pop_a > pop_b:
            # ● ERRORE 2: Aggiunta Sbagliata al Grafo
            self._graph.add_edge(a, b, weight=peso)

    # ... resto degli if ...
```

```
def buildGraph(self, genere):
```

```
    # ... creazione iniziale del grafo e riempimento di
    self._idMap ...

    mappa_popolarita = DAO.getPopolarita(genere)
    archi = DAO.getAllEdges(genere)
    for id1, id2 in archi:
        # ✔ CORREZIONE 1: Recupero gli Oggetti, non i
        numeri!

        # Uso la mappa che ho creato appositamente
        all'inizio del file
        nodo_A = self._idMap[id1]
        nodo_B = self._idMap[id2]

        # ✔ CORREZIONE 2: Prendo la popolarità usando
        solo .get()

        # Se l'id non esiste nel dizionario, .get restituisce 0
        senza fare crashare il programma
        pop_a = mappa_popolarita.get(id1, 0)
        pop_b = mappa_popolarita.get(id2, 0)
        peso = pop_a + pop_b

        # ✔ CORREZIONE 3: Passo gli Oggetti a NetworkX!
        if pop_a > pop_b:
            self._graph.add_edge(nodo_A, nodo_B,
            weight=peso)

        elif pop_a == pop_b:
            self._graph.add_edge(nodo_A, nodo_B,
            weight=peso)
```

La Maledizione del GROUP BY nelle Self-Join (SQL)

Questo è l'errore SQL più diffuso all'esame. Cerchiamo di capire visivamente cosa fa il database quando cerchi "Due artisti che hanno clienti in comune".

Come funziona il database in memoria:

Immagina che i Queen (ID 1) siano stati comprati da Marco, Luca e Anna.

Gli U2 (ID 2) sono stati comprati da Anna, Paolo e Luca.

Se nella sotto-query scrivi GROUP BY ArtistId, stai dicendo a MySQL: *"Schiaccia le righe e dammi un solo risultato per artista"*.

Cosa fa MySQL? Prende i Queen, vede che hanno 3 clienti, ne sceglie uno a caso (es. Marco) e butta via gli altri. Fa lo stesso con gli U2 (tiene Paolo).

Quando poi fai la Join (WHERE a1.cliente = a2.cliente), MySQL confronta Marco con Paolo. Non combaciano! E tu **perdi l'arco**, anche se in realtà avevano in comune Anna e Luca.

La Soluzione corretta:

Non devi mai raggruppare prima della Join! Devi usare DISTINCT per farti dare l'elenco pulito di *tutte* le coppie Artista-Cliente.

Solo **DOPO** averli uniti, usi il GROUP BY.

Il trucco del < (Minore):

Perché nella query si scrive a1.ArtistId < a2.ArtistId e non semplicemente != (diverso)?

Se usi !=, MySQL trova due archi per la stessa coppia

Ecco esattamente cosa ha fatto il tuo codice per calcolare la Popolarità, diviso nei 3 passaggi tecnici.

1. Il Calcolo in MySQL (Il file DAO.py)

Il testo chiedeva: *"La popolarità di un artista è la somma di tutti i brani acquistati"*.

I brani acquistati si trovano nella tabella invoiceline (le righe delle fatture), nella colonna Quantity.

Nel DAO hai scritto questa query:

```
SELECT ar.ArtistId, SUM(il.Quantity) as popolarita
```

```
FROM artist ar, album al, track t, genere g, invoiceline il, invoice ic
```

```
WHERE ... [join varie] ... AND g.Name = 'Rock'
```

```
GROUP BY ar.ArtistId
```

Cosa fa tecnicamente:

Il database prende tutti i brani 'Rock', guarda quante volte compaiono nelle fatture (invoiceline), e fa la somma (SUM) raggruppandoli per ogni singolo Artista (GROUP BY).

Il risultato netto che MySQL sputa fuori è una tabella a due colonne, tipo questa:

- ArtistId 1 (AC/DC) -> Popolarità 15
- ArtistId 2 (Aerosmith) -> Popolarità 8
- ArtistId 4 (Alanis Morissette) -> Popolarità 11

Sempre nel DAO, tu hai preso questa tabella SQL e l'hai trasformata in un **Dizionario Python**:

```
results = {}
```

for row in cursor:

```
results[row["ArtistId"]] = row["popolarita"]
```

results diventa: {1: 15, 2: 8, 4: 11}

2. L'Ottimizzazione nel file model.py (Evitare il Timeout)

Nel model.py, prima di iniziare a creare gli archi, hai chiamato il DAO **una volta sola**:

```
codePython
```

```
mappa_popolarita = DAO.getPopolarita(genere)
```

L'errore da non fare all'esame:

Se invece di creare questo dizionario tu avessi fatto una query al database per ogni singolo arco dentro al ciclo for (es. "dimmi la popolarità di AC/DC", poi "dimmi la popolarità di Aerosmith"), avresti fatto migliaia di query. Il programma si sarebbe bloccato per il limite di tempo.

Con il dizionario, hai scaricato tutti i dati in un colpo solo nella memoria RAM. Leggere dalla RAM ci mette 0.0001 secondi.

3. L'uso di .get(chiave, 0) per prevenire i Crash

Nel ciclo for degli archi, tu hai in mano due ID (id1 e id2) e ti servono i loro numeri di vendite.

Hai scritto questo:

```
codePython
```

```
pop_a = mappa_popolarita.get(id1, 0)
```

```
pop_b = mappa_popolarita.get(id2, 0)
```

Perché .get() e non mappa_popolarita[id1]?

Mettiamo caso che un artista Rock (es. ID 150) sia presente nel database, ma **nessun cliente ha mai comprato un suo brano**.

Dato che non c'è in nessuna fattura, la query SQL di prima lo ha ignorato. Nel dizionario, la chiave 150 non esiste.

- Se tu scrivi `mappa_popolarita[150]`, Python cerca la chiave, non la trova, e va in crash bloccando l'intero programma con un **KeyError**.
- Se tu scrivi `mappa_popolarita.get(150, 0)`, stai dicendo a Python: *"Cerca l'ID 150. Se non c'è nel dizionario, non crashare, ma restituiscimi semplicemente il valore 0"*.

In questo modo, gli artisti che non hanno venduto brani ricevono correttamente una popolarità di 0 senza rompere il codice.

COME LEGGERE GLI ERRORI PIU' COMUNI

File "C:\Users\chiar\PycharmProjects\2026-05-19-Simulazione1\database\DAO.py", line 83, in `getAllEdges`

```
results.append(row["id1"], row["id2"])
```

TypeError: list.append() takes exactly one argument (2 given)

Python vede due valori separati da una virgola e pensa che tu stia cercando di fare l'append di due cose contemporaneamente, cosa che la funzione append non sa fare.

Devi raggruppare i due ID in una singola "scatola" (una **tupla**), mettendo una coppia extra di parentesi tonde. In questo modo Python vede un solo oggetto (la tupla) che contiene i due ID.

1. KeyError: X

(L'errore più famoso dell'esame di TdP)

- **Cosa significa:** Hai cercato in un dizionario una chiave X che non esiste.
- **Dove capita di solito:** Nel **Model**, quando cerchi di estrarre le vendite o i brani in comune per un nodo: `vendite_A = mio_dizionario[nodoA.id]`.
- **Perché succede:** Perché il DAO, con la sua query SQL, non ha estratto quel nodo. Ad esempio, se `nodoA` non ha mai venduto nessun brano, COUNT lo salta. Il dizionario che ti arriva in Python non contiene l'ID di quel nodo.
- **Come si risolve in 1 secondo:** Non usare mai le parentesi quadre `[]` sui dizionari. Usa la funzione `.get()`, che ti permette di impostare un "valore di salvataggio" (come 0 o una lista vuota) se la chiave non esiste.
 - *Sbagliato:* `valore = dizionario[u.TrackId]`
 - *Corretto:* `valore = dizionario.get(u.TrackId, 0)` (se è un dizionario di numeri)
 - *Corretto:* `valore = dizionario.get(u.TrackId, set())` (se è un dizionario di insiemi)

2. TypeError: ... takes 2 positional arguments but 4 were given

- **Cosa significa:** Stai passando a una funzione più "parametri" di quanti ne possa accettare.
 - **Dove capita di solito:** Tra il **Controller** e il **Model** o tra il **Model** e il **DAO**.
 - **Perché succede:** Hai chiamato `self._model.buildGraph(genere, data_inizio, data_fine)` dal Controller, ma nel Model hai definito la funzione solo come `def buildGraph(self, genere):`. Hai dimenticato di aggiornare la firma della funzione.
 - **Come si risolve:** Vai a cercare la funzione che dà l'errore e aggiungi tra parentesi le variabili mancanti (non scordare il `self` come primo argomento se sei in una classe!).
-

3. **AttributeError: 'NoneType' object has no attribute 'X'**

- **Cosa significa:** Stai cercando di leggere un attributo (es. `.Name` o `.TrackId`) da una variabile che è vuota (`None`).
 - **Dove capita di solito:** Quando usi la **idMap** nel **Model** o quando cerchi di stampare qualcosa nel **Controller**.
 - **Perché succede:**
 - Caso A: Hai fatto una query SQL, non ha trovato nulla, e la funzione ha restituito `None` invece di una lista vuota `[]`.
 - Caso B: Hai cercato `self.idMap[id_sbagliato]` e ti ha restituito `None`.
 - **Come si risolve:** Controlla che le tue query SQL siano corrette e che le tue funzioni restituiscano sempre una lista (anche vuota) o che abbiano un `if risultato is None: return`.
-

4. **RecursionError: maximum recursion depth exceeded**

- **Cosa significa:** Il tuo algoritmo ricorsivo (il Punto 2 dell'esame) sta chiamando se stesso all'infinito e Python ha fermato l'esecuzione per evitare di bloccare il computer.
 - **Dove capita di solito:** Nel **Model**, nel metodo `_ricorsione(...)`.
 - **Perché succede:** Non hai gestito bene i "casi di stop". Le cause principali sono:
 1. Hai dimenticato il `return` dentro il Caso Base.
 2. Non hai messo il controllo per evitare i "cicli infiniti" (es. `if vicino not in parziale:`). Se A va in B, e B va in A, il programma salterà tra A e B all'infinito.
 - **Come si risolve:** Controlla la riga subito dopo il `for vicino in self.grafo...` e assicurati di avere scritto `if vicino not in parziale:`.
-

5. **TypeError: 'dict' object is not callable**

- **Cosa significa:** Stai usando delle parentesi tonde `()` su un dizionario, come se fosse una funzione.
 - **Dove capita di solito:** Nel **Model** o nel **DAO**.
 - **Perché succede:** È un errore di distrazione! In Python le chiavi dei dizionari si leggono con le parentesi quadre `[]` (o con `.get()`), non con quelle tonde.
 - *Sbagliato:* `valore = mio_dizionario(id_nodo)`
 - *Corretto:* `valore = mio_dizionario[id_nodo]` o `mio_dizionario.get(id_nodo)`
-

7. **NetworkXError: The node N is not in the graph**

- **Cosa significa:** Stai cercando di creare un arco tra due nodi (o di calcolare i vicini di un nodo), ma uno di quei nodi non esiste nel grafo.

- **Dove capita di solito:** Nel **Model** (buildGraph).
- **Perché succede:** Hai estratto l'elenco degli archi dal DAO (es. la lista di tuple con la *Super Query* (A, B, peso)) e stai facendo `self.grafo.add_edge(A, B)`. Ma non hai prima aggiunto A e B usando `self.grafo.add_node()`.
- **Come si risolve:** Nel metodo `buildGraph`, la *primissima cosa* che devi fare è sempre interrogare il DAO per ottenere la lista di tutti i nodi singoli e usare un ciclo `for`:

codePython

```
nodi = DAO.getAllNodi()
```

```
for n in nodi:
```

```
    self.grafo.add_node(n)
```

Solo *dopo* puoi iniziare ad aggiungere gli archi.

8. L'errore del Grafo non svuotato ("Falsi nodi infiniti")

- **Cosa significa:** L'applicazione non crasha, ma quando premi "Crea Grafo" più di una volta di seguito (magari cambiando il genere o la nazione nel menù a tendina), il **numero di Nodi e Archi continua ad aumentare** invece di calcolare quelli nuovi.
- **Dove capita di solito:** Nel **Model** (buildGraph).
- **Perché succede:** NetworkX non cancella automaticamente il grafo precedente quando richiami la funzione. Si limiterà ad aggiungere i nuovi nodi a quelli della ricerca precedente.
- **Come si risolve:** La primissima riga di codice dentro `buildGraph` deve **sempre** essere:

codePython

```
self.grafo.clear()
```

```
self.idMap.clear() # Se usi la mappa degli ID, svuota anche lei!
```

9. `mysql.connector.errors.ProgrammingError: 1064 (42000): You have an error in your SQL syntax`

- **Cosa significa:** Hai sbagliato a scrivere la query SQL.
- **Dove capita di solito:** Nel **DAO**.
- **Perché succede (i 3 motivi classici dell'esame):**

1. Hai dimenticato di inserire uno spazio a fine riga in una stringa multiline:

codePython

```
query = "SELECT * FROM track" +
```

```
"WHERE GenreId = 1" # Diventa "trackWHERE" (Manca lo spazio!)
```


2. Hai usato i doppi apici "" all'interno di una query dichiarata con i doppi apici. In SQL usa sempre e solo il singolo apice ' per le stringhe (es. WHERE Name = 'Jazz').
 3. Hai dimenticato di mettere una virgola tra le tabelle nel FROM o c'è un errore di battitura (JOIN senza ON).
- **Come si risolve:** Vai su DBeaver/HeidiSQL, incolla la tua query, scrivi un valore di test al posto del %s e provala. Risolvi l'errore lì e poi copiala in Python.
-

Se quando stampo credo di aver fatto tutto giusto ma mi stampa 0 nodi:

1) controlla di aver messo la riga nel controller : `self._model.buildGraph(...)`

2) **Il DAO restituisce una lista vuota? (L'errore nella Query SQL)**

È il motivo nel 90% dei casi. Stai chiedendo i nodi a SQL, ma lui ti sta rispondendo con il vuoto.

Cosa fare: Vai in modalità "debug rustico". Prima di `self.grafo.add_node()`, stampa la lista che ti arriva dal DAO.

```
codePython
```

```
nodi = DAO.getNodi(parametro)
```

```
print(f"DEBUG: Ho estratto dal DAO {len(nodi)} nodi.")
```

3) **Hai dimenticato **row nella creazione dell'oggetto?**

4) ricordati il dDown : `paese = self._view._ddNazione.value` #value senza parentesi

Se ho 0 Archi:

1. **Il Dizionario è vuoto o non matcha:** Il DAO ha fatto la mega-query (es. le vendite), ma nel Model fai `dizionario.get(u.id)`. Assicurati che l'id che stai passando sia dello stesso tipo (intero o stringa) di quello salvato nel dizionario.
2. **Le regole nel doppio FOR sono troppo rigide:** La traccia dice "differenza <= 5". Forse nei tuoi dati nessuno ha una differenza così piccola? Stampa sempre una volta `print(f"Confronto {val_u} con {val_v}")` per vedere i valori che il Python sta calcolando.
3. **Hai dimenticato l'add_edge():** Sembra una banalità, ma a volte si scivola sugli spazi (l'indentazione in Python). L'istruzione `self.grafo.add_edge(u, v)` potrebbe essere finita fuori dall'if o indentata male.